Prácticas de SQL

100 ejercicios resueltos con PostgreSQL

Nivel básico - Experto

Dataset: Brazilian E-Commerce Public Dataset

by Olist (Kaggle)

Josefina Urquiza

1. Introducción

¿Por qué este workbook?

Este documento reúne 100 ejercicios de SQL desarrollados utilizando el Brazilian E-Commerce Public Dataset by Olist, uno de los datasets públicos más utilizados para practicar análisis de datos.

Los ejercicios fueron diseñados con una dificultad progresiva, comenzando con consultas básicas y avanzando hacia problemas similares a los que pueden aparecer en entrevistas técnicas o proyectos reales de análisis de datos.

Todas las soluciones están escritas en PostgreSQL e incluyen consultas que utilizan desde operaciones simples de filtrado y agregación hasta funciones ventana, CTEs y análisis más avanzados.

2. Sobre el dataset

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle)

El Brazilian E-Commerce Public Dataset by Olist contiene información anonimizada de aproximadamente 100.000 pedidos realizados entre 2016 y 2018 en el marketplace brasileño Olist.

Los datos se encuentran normalizados en múltiples tablas relacionadas entre sí, permitiendo analizar distintos aspectos del negocio, como clientes, pedidos, productos, vendedores, pagos y reseñas.

Este tipo de estructura es similar a la que puede encontrarse en bases de datos utilizadas en entornos reales de e-commerce.

Tablas principals (format csv):

- ***orders***: order_id, customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date
- ***customers***: customer_id, customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
- ***order_items:*** order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value
- olist_order_payments_dataset: order_id, payment_sequential, payment_type, payment_installments, payment_value
- ***order_reviews***: review_id, order_id, review_score, review_comment_title, review_comment_message, review_creation_date
- ***products*****:** product_id, product_category_name, product_weight_g, product_length_cm, product_height_cm, product_width_cm
- ***sellers***: seller_id, seller_zip_code_prefix, seller_city, seller_state
- ***geolocation***: geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state product_category_name_translation: product_category_name, product_category_name_english

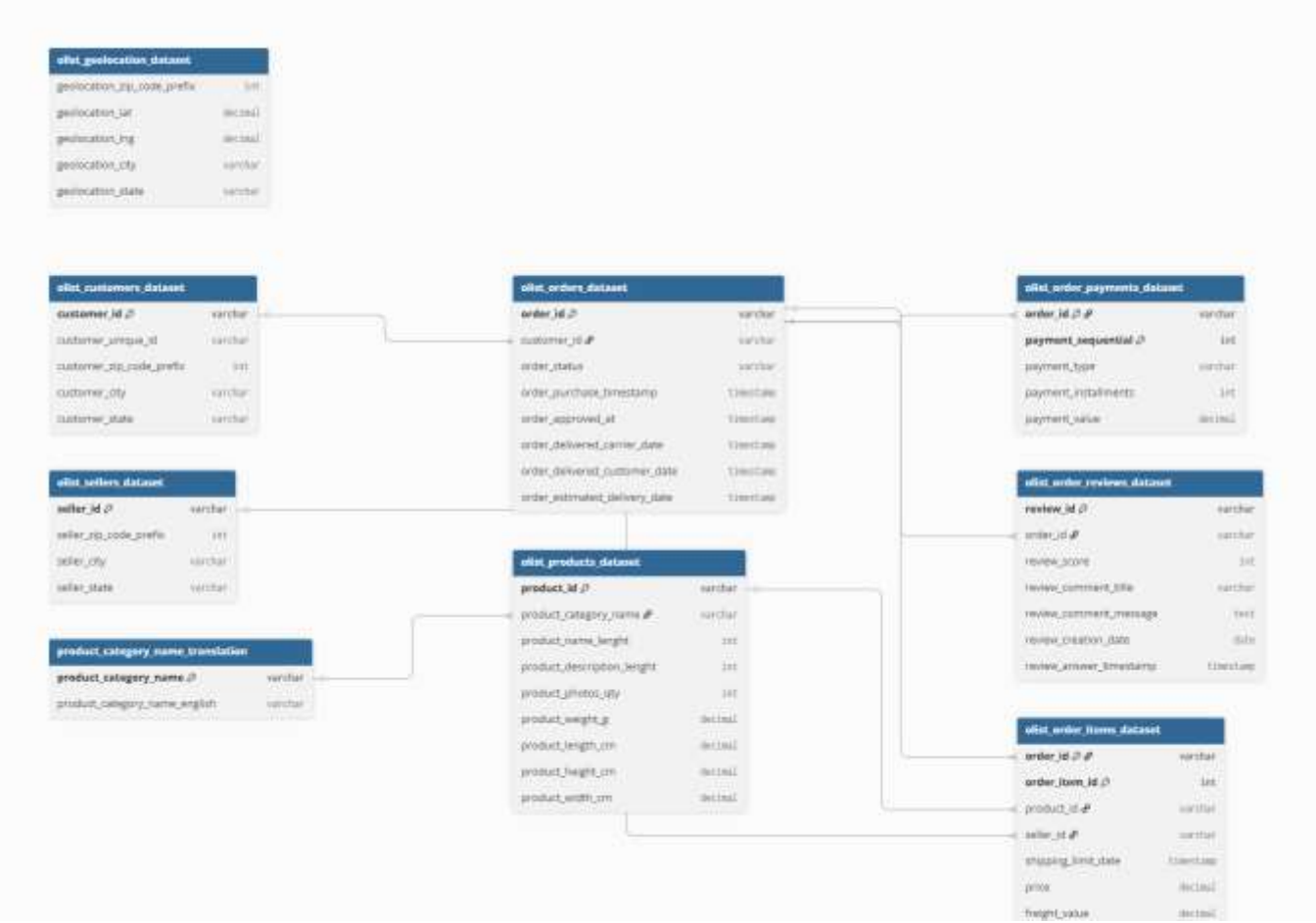
Importante: customer_id es distinto para cada pedido de un mismo cliente; para identificar clientes únicos hay que usar customer_unique_id.

Consideraciones

Todas las consultas de este workbook fueron desarrolladas utilizando PostgreSQL.

Si utilizás otro gestor de bases de datos (MySQL, SQL Server, SQLite, BigQuery, etc.), es posible que algunas funciones deban adaptarse, especialmente aquellas relacionadas con fechas y funciones ventana.

3. Modelo de datos



4. ¿Qué vas a practicar?

Tema	Nivel
SELECT	Básico
WHERE	Básico
ORDER BY	Básico

Tema	Nivel
GROUP BY	Intermedio
HAVING	Intermedio
JOIN	Intermedio
Subconsultas	Avanzado
CTE	Avanzado
CASE	Intermedio
Funciones de fecha	Intermedio
Window Functions	Experto
Ranking	Experto
LAG / LEAD	Experto
NTILE	Experto
RFM	Experto
Cohortes	Experto

5. A quien esta dirigido?

Este workbook está pensado para estudiantes, analistas de datos y cualquier persona que quiera mejorar sus habilidades en SQL utilizando un dataset real de e-commerce. Los ejercicios avanzan progresivamente desde consultas básicas hasta problemas de análisis que simulan situaciones habituales en proyectos de negocio y entrevistas técnicas.

6. Recomendaciones

Cómo aprovechar este workbook

- Leer únicamente el enunciado antes de comenzar.
- Intentar resolver el problema sin consultar la solución.
- Comparar la propia consulta con la propuesta.
- Analizar si existen alternativas más eficientes.
- Revisar el plan de ejecución (EXPLAIN) cuando sea posible.
- Probar pequeñas modificaciones para reforzar el aprendizaje.

Nivel 1 Básico

Consultas a practicar: SELECT, WHERE, ORDER BY, LIMIT, DISTINCT

1. ***¿Cuántos pedidos hay en total en la tabla de pedidos?***

Consulta SQL:

```
SELECT COUNT(*) AS total_ordenes
FROM olist_orders_dataset;
```

2. ***Listar los primeros 10 pedidos con su estado y fecha de compra, ordenados por fecha de compra de forma descendente.***

Consulta SQL:

```
SELECT order_id, order_status, order_purchase_timestamp
FROM olist_orders_dataset
ORDER BY order_purchase_timestamp DESC
LIMIT 10;
```

3. ***¿Cuáles son los distintos valores posibles de order_status?***

Consulta SQL:

```
SELECT DISTINCT order_status
FROM olist_orders_dataset;
```

4. ***Mostrar todos los pedidos que fueron cancelados.***

Consulta SQL:

```
SELECT *
FROM olist_orders_dataset
WHERE order_status = 'canceled';
```

5. ***¿Cuántos clientes distintos (personas reales, no pedidos) hay registrados?***

Consulta SQL:

```
SELECT COUNT(DISTINCT customer_unique_id) AS clientes_unicos
FROM olist_customers_dataset;
```

Nota: *Se usa customer_unique_id y no customer_id, porque cada pedido genera un customer_id distinto.*

6. ***Listar las combinaciones distintas de ciudad y estado donde hay vendedores registrados, ordenadas alfabéticamente.***

Consulta SQL:

```
SELECT DISTINCT seller_city, seller_state
FROM olist_sellers_dataset
ORDER BY seller_state, seller_city;
```

7. ***Cuántos pedidos fueron entregados?.***

Consulta SQL:

```
SELECT COUNT(*) AS pedidos_entregados
FROM olist_orders_dataset
WHERE order_status='delivered';
```

8. Cuántos tipos distintos de pago existen?

Consulta SQL:

```
SELECT COUNT(DISTINCT payment_type) AS tipos_pago
FROM olist_order_payments_dataset;
```

9. Listar 20 productos con mayor precio

Consulta SQL:

```
SELECT product_id, price
FROM olist_order_items_dataset
ORDER BY price DESC
LIMIT 20;
```

10. ¿Cuántos productos distintos (product_id) aparecen en la tabla de order_items?

Consulta SQL:

```
SELECT COUNT(DISTINCT product_id) AS productos_distintos
FROM olist_order_items_dataset;
```

11. Listar los vendedores por estado

Consulta SQL:

```
SELECT seller_state, COUNT(*) AS vendedores
FROM olist_sellers_dataset
GROUP BY seller_state
ORDER BY vendedores DESC;
```

12. *Listar las categorías en inglés*

Consulta SQL:

```
SELECT DISTINCT product_category_name_english
FROM product_category_name_translation
ORDER BY product_category_name_english;
```

13. *Listar los 10 estados (customer_state) con más pedidos registrados.*

Consulta SQL:

```
SELECT c.customer_state, COUNT(o.order_id) AS cantidad_pedidos
FROM olist_customers_dataset c
```

```
JOIN olist_orders_dataset o
ON c.customer_id = o.customer_id
GROUP BY c.customer_state
ORDER BY cantidad_pedidos DESC
LIMIT 10;
```

14. Pedidos con entrega estimada antes del 2018-01-01

Consulta SQL:

```
SELECT *
FROM olist_orders_dataset
WHERE order_estimated_delivery_date < '2018-01-01';
```

15. Precio mínimo y máximo

Consulta SQL:

```
SELECT MIN(price) AS precio_minimo, MAX(price) AS precio_maximo
FROM olist_order_items_dataset;
```

16. Cantidad de pedidos con review 5

Consulta SQL:

```
SELECT COUNT(*) AS total
FROM olist_order_reviews_dataset
WHERE review_score=5;
```

17. Precio promedio por vendedor

Consulta SQL:

```
SELECT seller_id, ROUND(AVG(price),2) AS precio_promedio
FROM olist_order_items_dataset
GROUP BY seller_id
ORDER BY precio_promedio DESC;
```

18. ¿Cuántos pedidos NO fueron entregados (status distinto de 'delivered')?

Consulta SQL:

```
SELECT COUNT(*) AS pedidos_no_entregados
FROM olist_orders_dataset
WHERE order_status <> 'delivered';
```

19. ¿Cuál es la fecha de compra más antigua y la más reciente registradas?

Consulta SQL:

```
SELECT MIN(order_purchase_timestamp) AS primera_compra,
```

```
    MAX(order_purchase_timestamp) AS ultima_compra
FROM olist_orders_dataset;
```

20. Listar los 15 productos con mayor peso (product_weight_g).

Consulta SQL:

```
SELECT product_id, product_weight_g
FROM olist_products_dataset
ORDER BY product_weight_g DESC
LIMIT 15;
```

21. ¿Cuántas reviews tienen un comentario escrito (review_comment_message no es NULL)?

Consulta SQL:

```
SELECT COUNT(*) AS reviews_con_comentario
FROM olist_order_reviews_dataset
WHERE review_comment_message IS NOT NULL;
```

22. Listar los pedidos cuyo estado sea 'shipped' o 'processing'.

Consulta SQL:

```
SELECT order_id, order_status, order_purchase_timestamp
FROM olist_orders_dataset
WHERE order_status IN ('shipped', 'processing');
```

23. ¿Cuántos pagos se hicieron con más de 1 cuota (payment_installments > 1)?

Consulta SQL:

```
SELECT COUNT(*) AS pagos_en_cuotas
FROM olist_order_payments_dataset
WHERE payment_installments > 1;
```

24. Listar las 10 ciudades de vendedores con más registros, junto con la cantidad.

Consulta SQL:

```
SELECT seller_city, COUNT(*) AS cantidad_vendedores
FROM olist_sellers_dataset
GROUP BY seller_city
ORDER BY cantidad_vendedores DESC
LIMIT 10;
```

25. ¿Cuál es el valor de flete (freight_value) promedio y máximo de todos los items vendidos?

Consulta SQL:

```
SELECT ROUND(AVG(freight_value), 2) AS flete_promedio,
```

```
    MAX(freight_value) AS flete_maximo
FROM olist_order_items_dataset;
```

Nivel 2 Intermedio

Consultas a practicar: JOIN, GROUP BY, HAVING, funciones de agregación

1. ***Pedidos con más de un producto***

Consulta SQL:

```
SELECT order_id, COUNT(*) AS cantidad_productos
FROM olist_order_items_dataset
GROUP BY order_id
HAVING COUNT(*)>1;
```

2. ***¿Cuál es el precio promedio de los productos vendidos, por categoría (en inglés)? Ordenar de mayor a menor.***

Consulta SQL:

```
SELECT t.product_category_name_english, ROUND(AVG(oi.price), 2) AS precio_promedio
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY precio_promedio DESC;
```

3. ***¿Cuántos pedidos hizo cada cliente? Mostrar solo los clientes que hicieron más de un pedido.***

Consulta SQL:

```
SELECT c.customer_unique_id, COUNT(DISTINCT o.order_id) AS cantidad_pedidos
FROM olist_orders_dataset o
JOIN olist_customers_dataset c ON o.customer_id = c.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(DISTINCT o.order_id) > 1
ORDER BY cantidad_pedidos DESC;
```

4. ***¿Cuál es el estado (customer_state) con más clientes únicos?***

Consulta SQL:

```
SELECT customer_state, COUNT(DISTINCT customer_unique_id) AS clientes
FROM olist_customers_dataset
GROUP BY customer_state
ORDER BY clientes DESC
LIMIT 1;
```

5. ***¿Cuál es el monto total pagado por tipo de método de pago?***

Consulta SQL:

```
SELECT payment_type, ROUND(SUM(payment_value), 2) AS total_pagado
FROM olist_order_payments_dataset
GROUP BY payment_type
ORDER BY total_pagado DESC;
```

6. ***¿Cuál es la calificación promedio (review_score) por categoría de producto? Ordenar de la peor calificada a la mejor.***

Consulta SQL:

```
SELECT t.product_category_name_english, ROUND(AVG(r.review_score), 2) AS calificacion_promedio
FROM olist_order_reviews_dataset r
JOIN olist_orders_dataset o ON r.order_id = o.order_id
JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY calificacion_promedio ASC;
```

7. ***¿Cuántos pedidos entregados hubo por mes y año?***

Consulta SQL:

```
SELECT DATE_TRUNC('month', order_purchase_timestamp) AS mes, COUNT(*) AS pedidos
FROM olist_orders_dataset
WHERE order_status = 'delivered'
GROUP BY mes
ORDER BY mes;
```

8. ***¿Cuáles son los 10 vendedores con mayor facturación total (suma de price)?***

Consulta SQL:

```
SELECT seller_id, ROUND(SUM(price), 2) AS facturacion_total
FROM olist_order_items_dataset
GROUP BY seller_id
ORDER BY facturacion_total DESC
LIMIT 10;
```

9. ***¿Cuál es el costo de envío (freight_value) promedio, agrupado por estado del cliente?***

Consulta SQL:

```
SELECT c.customer_state, ROUND(AVG(oi.freight_value), 2) AS flete_promedio
FROM olist_order_items_dataset oi
JOIN olist_orders_dataset o ON oi.order_id = o.order_id
JOIN olist_customers_dataset c ON o.customer_id = c.customer_id
GROUP BY c.customer_state
```

```
ORDER BY flete_promedio DESC;
```

10. Listar los pedidos del año 2017

Consulta SQL:

```
SELECT *
FROM olist_orders_dataset
WHERE EXTRACT(YEAR FROM order_purchase_timestamp)=2017;
```

11. Mostrar los 10 clientes (customer_unique_id) que más dinero gastaron en total. Mostrar: customer_unique_id, cantidad de pedidos, dinero gastado Ordenar de mayor a menor.

Consulta SQL:

```
SELECT
   c.customer_unique_id,
   COUNT(DISTINCT o.order_id) AS cantidad_pedidos,
   SUM(op.payment_value) AS total_gastado
FROM customers c
JOIN orders o
   ON c.customer_id = o.customer_id
JOIN olist_order_payments_dataset op
   ON o.order_id = op.order_id
GROUP BY c.customer_unique_id
ORDER BY total_gastado DESC
LIMIT 10;
```

12. ¿Cuál fue el ticket promedio (payment_value) por estado del cliente? Mostrar: Estado, Cantidad de pedidos, Ticket promedio Ordenar por ticket promedio descendente.

Consulta SQL:

```
SELECT
   c.customer_state,
   COUNT(o.order_id) AS pedidos,
   ROUND(AVG(op.payment_value),2) AS ticket_promedio
FROM customers c
JOIN orders o
   ON c.customer_id=o.customer_id
JOIN olist_order_payments_dataset op
   ON o.order_id=op.order_id
GROUP BY c.customer_state
ORDER BY ticket_promedio DESC;
```

13. Calcular cuánto tardó cada pedido en entregarse. Mostrar únicamente aquellos pedidos cuya entrega demoró más de 30 días.

Consulta SQL:

```
SELECT
   order_id,
   order_purchase_timestamp,
   order_delivered_customer_date,
   (order_delivered_customer_date::date -
    order_purchase_timestamp::date) AS dias_entrega
FROM orders
WHERE order_delivered_customer_date IS NOT NULL
AND (order_delivered_customer_date::date -
    order_purchase_timestamp::date) > 30
ORDER BY dias_entrega DESC;
```

14. ¿Cuáles son las 5 categorías de productos más vendidas (en cantidad de items)?
Consulta SQL:

```
SELECT t.product_category_name_english, COUNT(*) AS cantidad_vendida
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY cantidad_vendida DESC
LIMIT 5;
```

15. ¿Cuántos vendedores distintos vendieron cada categoría de producto? Ordenar de mayor a menor.
Consulta SQL:

```
SELECT t.product_category_name_english, COUNT(DISTINCT oi.seller_id) AS vendedores_distintos
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY vendedores_distintos DESC;
```

16. ¿ Cuál es la cantidad de pedidos y el porcentaje de pedidos cancelados sobre el total, por año?
Consulta SQL:

```
SELECT
   EXTRACT(YEAR FROM order_purchase_timestamp) AS anio,
   COUNT(*) AS total_pedidos,
   SUM(CASE WHEN order_status = 'canceled' THEN 1 ELSE 0 END) AS pedidos_cancelados,
   ROUND(
```

```
    100.0 * SUM(CASE WHEN order_status = 'canceled' THEN 1 ELSE 0 END) / COUNT(*),
    2
  ) AS porcentaje_cancelados
FROM olist_orders_dataset
GROUP BY anio
ORDER BY anio;
```

17. ***Mostrar los vendedores que tuvieron una calificación promedio menor a 3 (considerando las reviews de los pedidos en los que participaron), junto con la cantidad de reviews que tienen.***

Consulta SQL:

```
SELECT
  oi.seller_id,
  COUNT(r.review_id) AS cantidad_reviews,
  ROUND(AVG(r.review_score), 2) AS calificacion_promedio
FROM olist_order_items_dataset oi
JOIN olist_order_reviews_dataset r ON oi.order_id = r.order_id
GROUP BY oi.seller_id
HAVING AVG(r.review_score) < 3
ORDER BY calificacion_promedio ASC;
```

18. ***¿Cuál es la cantidad de pedidos por método de pago y el promedio de cuotas (payment_installments) usado en cada uno?.***

Consulta SQL:

```
SELECT
  payment_type,
  COUNT(*) AS cantidad_pedidos,
  ROUND(AVG(payment_installments), 2) AS promedio_cuotas
FROM olist_order_payments_dataset
GROUP BY payment_type
ORDER BY cantidad_pedidos DESC
```

19. ***Listar las 10 ciudades de clientes con mayor gasto total, mostrando también la cantidad de clientes únicos de esa ciudad.***

Consulta SQL:

```
SELECT
  c.customer_city,
  COUNT(DISTINCT c.customer_unique_id) AS clientes_unicos,
  ROUND(SUM(op.payment_value), 2) AS gasto_total
FROM olist_customers_dataset c
JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
JOIN olist_order_payments_dataset op ON o.order_id = op.order_id
```

```
GROUP BY c.customer_city
ORDER BY gasto_total DESC
LIMIT 10;
```

20. ***¿Cuál es el peso promedio (product_weight_g) de los productos por categoría, solo para categorías con más de 100 productos vendidos?.***

Consulta SQL:

```
SELECT
   t.product_category_name_english,
   COUNT(*) AS cantidad_vendida,
   ROUND(AVG(p.product_weight_g), 2) AS peso_promedio
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
HAVING COUNT(*) > 100
ORDER BY peso_promedio DESC;
```

21. ***¿Cuántos pedidos tuvieron más de un método de pago asociado (pagos combinados)?***

Consulta SQL:

```
SELECT COUNT(*) AS pedidos_con_pago_combinado
FROM (
   SELECT order_id
   FROM olist_order_payments_dataset
   GROUP BY order_id
   HAVING COUNT(DISTINCT payment_type) > 1
) sub;
```

22. ***¿Cuál es la diferencia promedio (en días) entre la fecha estimada de entrega y la fecha real de entrega, por estado del cliente? Ordenar mostrando primero los estados donde más se atrasan las entregas.***

Consulta SQL:

```
SELECT
   c.customer_state,
   ROUND(AVG(
      (o.order_delivered_customer_date::date - o.order_estimated_delivery_date::date)
   ), 2) AS dias_diferencia_promedio
FROM olist_orders_dataset o
JOIN olist_customers_dataset c ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
```

```
ORDER BY dias_diferencia_promedio DESC;si
```

23. ¿Cuál es el vendedor con mayor cantidad de pedidos distintos entregados?

Consulta SQL:

```
SELECT oi.seller_id, COUNT(DISTINCT o.order_id) AS pedidos_entregados
FROM olist_order_items_dataset oi
JOIN olist_orders_dataset o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY oi.seller_id
ORDER BY pedidos_entregados DESC
LIMIT 1;
```

24. ¿Cuántas categorías de producto tienen un precio promedio mayor a 200?

Consulta SQL:

```
SELECT COUNT(*) AS categorias_caras
FROM (
   SELECT t.product_category_name_english, AVG(oi.price) AS precio_promedio
   FROM olist_order_items_dataset oi
   JOIN olist_products_dataset p ON oi.product_id = p.product_id
   JOIN product_category_name_translation t
     ON p.product_category_name = t.product_category_name
   GROUP BY t.product_category_name_english
   HAVING AVG(oi.price) > 200
) sub;
```

25. Por año, ¿cuál es el porcentaje de pagos hechos con 'boleto' vs 'credit_card'?

Consulta SQL:

```
SELECT
   EXTRACT(YEAR FROM o.order_purchase_timestamp) AS anio,
   ROUND(100.0 * SUM(CASE WHEN op.payment_type = 'boleto' THEN 1 ELSE 0 END) / COUNT(*), 2)
AS pct_boleto,
   ROUND(100.0 * SUM(CASE WHEN op.payment_type = 'credit_card' THEN 1 ELSE 0 END) / COUNT(*),
2) AS pct_tarjeta
FROM olist_order_payments_dataset op
JOIN olist_orders_dataset o ON op.order_id = o.order_id
GROUP BY anio
ORDER BY anio;
```

Nivel 3 - Avanzado

Consultas a practicar: Subconsultas, CTE (WITH), fechas y funciones de tiempo

1. ***Calcular el tiempo promedio (en días) entre la fecha de compra y la fecha de entrega real, solo para pedidos entregados.***

Consulta SQL:

```
SELECT ROUND(AVG(
  EXTRACT(EPOCH FROM (order_delivered_customer_date - order_purchase_timestamp)) / 86400
), 1) AS dias_promedio_entrega
FROM olist_orders_dataset
WHERE order_status = 'delivered'
  AND order_delivered_customer_date IS NOT NULL;
```

2. ***¿Qué porcentaje de los pedidos entregados llegó después de la fecha estimada de entrega?***

Consulta SQL:

```
SELECT ROUND(
  100.0 * SUM(CASE WHEN order_delivered_customer_date > order_estimated_delivery_date
                THEN 1 ELSE 0 END) / COUNT(*)
, 2) AS porcentaje_atrasados
FROM olist_orders_dataset
WHERE order_status = 'delivered'
  AND order_delivered_customer_date IS NOT NULL;
```

3. ***Usando una CTE (WITH), calcular el ticket promedio por pedido y mostrar los pedidos cuyo monto total supera ese promedio.***

Consulta SQL:

```
WITH ticket_por_pedido AS (
  SELECT order_id, SUM(payment_value) AS monto_total
  FROM olist_order_payments_dataset
  GROUP BY order_id
),
promedio AS (
  SELECT AVG(monto_total) AS promedio_general
  FROM ticket_por_pedido
)
SELECT tp.order_id, tp.monto_total
FROM ticket_por_pedido tp, promedio pr
WHERE tp.monto_total > pr.promedio_general
ORDER BY tp.monto_total DESC;
```

4. ***Encontrar los productos que nunca recibieron una reseña.***

Consulta SQL:

```
SELECT p.product_id
FROM olist_products_dataset p
WHERE NOT EXISTS (
  SELECT 1
  FROM olist_order_items_dataset oi
  JOIN olist_order_reviews_dataset r ON oi.order_id = r.order_id
  WHERE oi.product_id = p.product_id
);
```

5. ***¿Cuál es la categoría de producto con mayor cantidad de reseñas de 1 estrella?***

Consulta SQL:

```
SELECT t.product_category_name_english, COUNT(*) AS cantidad_1_estrella
FROM olist_order_reviews_dataset r
JOIN olist_order_items_dataset oi ON r.order_id = oi.order_id
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
WHERE r.review_score = 1
GROUP BY t.product_category_name_english
ORDER BY cantidad_1_estrella DESC
LIMIT 1;
```

6. ***Calcular, por mes, la cantidad de pedidos y el porcentaje de variación respecto al mes anterior.***

Consulta SQL:

```
WITH pedidos_mes AS (
  SELECT DATE_TRUNC('month', order_purchase_timestamp) AS mes, COUNT(*) AS cantidad
  FROM olist_orders_dataset
  GROUP BY mes)
SELECT mes, cantidad,
  ROUND(100.0 * (cantidad - LAG(cantidad) OVER (ORDER BY mes))
      / LAG(cantidad) OVER (ORDER BY mes), 2) AS variacion_pct
FROM pedidos_mes
ORDER BY mes;
```

Nota: *Ya usa una función de ventana (LAG) como puente hacia el nivel experto.*

7. ***¿Qué clientes compraron productos de más de una categoría distinta?***

Consulta SQL:

```
SELECT c.customer_unique_id, COUNT(DISTINCT t.product_category_name_english) AS
categorias_distintas
```

```
FROM olist_customers_dataset c
JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY c.customer_unique_id
HAVING COUNT(DISTINCT t.product_category_name_english) > 1
ORDER BY categorias_distintas DESC;
```

8. Usando una subconsulta correlacionada, mostrar el pedido de mayor monto total de cada cliente.

Consulta SQL:

```
SELECT c.customer_unique_id, o.order_id, tp.monto_total
FROM olist_customers_dataset c
JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
JOIN (
  SELECT order_id, SUM(payment_value) AS monto_total
  FROM olist_order_payments_dataset
  GROUP BY order_id
) tp ON o.order_id = tp.order_id
WHERE tp.monto_total = (
  SELECT MAX(tp2.monto_total)
  FROM olist_orders_dataset o2
  JOIN (
    SELECT order_id, SUM(payment_value) AS monto_total
    FROM olist_order_payments_dataset
    GROUP BY order_id
  ) tp2 ON o2.order_id = tp2.order_id
  WHERE o2.customer_id = o.customer_id
);
```

9. Obtener el ranking de vendedores según su facturación. Mostrar: seller_id, ventas, ranking

Utilizar una función ventana (DENSE_RANK)

Consulta SQL:

```
SELECT
    seller_id,
    SUM(price) AS ventas,
    DENSE_RANK() OVER(
        ORDER BY SUM(price) DESC
    ) AS ranking
FROM order_items
```

```
GROUP BY seller_id;
```

10. ***Encontrar la categoría de producto con mejor calificación promedio. Considerar solamente categorías con al menos 100 reviews.***

Consulta SQL:

```
SELECT
   pct.product_category_name_english,
   ROUND(AVG(r.review_score),2) AS promedio_review,
   COUNT(*) AS cantidad_reviews
FROM order_reviews r
JOIN order_items oi
   ON r.order_id=oi.order_id
JOIN products p
   ON oi.product_id=p.product_id
JOIN product_category_name_translation pct
   ON p.product_category_name=pct.product_category_name
GROUP BY pct.product_category_name_english
HAVING COUNT(*)>=100
ORDER BY promedio_review DESC;
```

11. ***Para cada estado del cliente obtener el vendedor que más facturó. Debe aparecer un solo vendedor por estado.***

Consulta SQL:

```
WITH ventas AS (
   SELECT
      c.customer_state,
      oi.seller_id,
      SUM(oi.price) AS ventas
   FROM customers c
   JOIN orders o
      ON c.customer_id = o.customer_id
   JOIN order_items oi
      ON o.order_id = oi.order_id
   GROUP BY
      c.customer_state,
      oi.seller_id
)

SELECT
   customer_state,
   seller_id,
   ventas
```

```
FROM (
   SELECT
      customer_state,
      seller_id,
      ventas,
      ROW_NUMBER() OVER (
         PARTITION BY customer_state
         ORDER BY ventas DESC
      ) AS rn
   FROM ventas
) ranking
WHERE rn = 1
ORDER BY customer_state;
```

12. Mostrar la evolución mensual de las ventas junto con el acumulado histórico.

Consulta SQL:

```
SELECT
   DATE_TRUNC('month',order_purchase_timestamp) mes,
   SUM(payment_value) ventas,
   SUM(SUM(payment_value))
      OVER(
      ORDER BY DATE_TRUNC('month',order_purchase_timestamp)
      ) ventas_acumuladas
FROM orders o
JOIN olist_order_payments_dataset op
ON o.order_id=op.order_id
GROUP BY mes
ORDER BY mes;
```

13. Mostrar el % de pedidos interestatales (vendedor y comprador en distinto estado).

Consulta SQL:

```
SELECT ROUND(100.0 * SUM(CASE WHEN c.customer_state <> s.seller_state THEN 1 ELSE 0 END)
     / COUNT(*), 2) AS pct_interestatal
FROM olist_orders_dataset o
JOIN olist_customers_dataset c ON o.customer_id = c.customer_id
JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id;
```

14. Mostrar por mes, cantidad de pedidos de clientes nuevos vs. recurrentes (subconsulta).

Consulta SQL:

```
WITH primera_compra AS (
  SELECT c.customer_unique_id, MIN(o.order_purchase_timestamp) AS primera_fecha
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  GROUP BY c.customer_unique_id
)
SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS mes,
  SUM(CASE WHEN o.order_purchase_timestamp = pc.primera_fecha THEN 1 ELSE 0 END) AS pedidos_nuevos,
  SUM(CASE WHEN o.order_purchase_timestamp <> pc.primera_fecha THEN 1 ELSE 0 END) AS pedidos_recurrentes
FROM olist_orders_dataset o
JOIN olist_customers_dataset c ON o.customer_id = c.customer_id
JOIN primera_compra pc ON c.customer_unique_id = pc.customer_unique_id
GROUP BY mes
ORDER BY mes;
```

15. ***Mostrar items cuyo flete (freight_value) es mayor al precio del producto.***

Consulta SQL:

```
SELECT order_id, product_id, price, freight_value
FROM olist_order_items_dataset
WHERE freight_value > price
ORDER BY (freight_value - price) DESC;
```

16. Mostrar la cantidad y % de reviews por puntaje (CTE).

Consulta SQL:

```
WITH total AS (
  SELECT COUNT(*) AS total_reviews FROM olist_order_reviews_dataset
)
SELECT r.review_score, COUNT(*) AS cantidad,
  ROUND(100.0 * COUNT(*) / t.total_reviews, 2) AS porcentaje
FROM olist_order_reviews_dataset r, total t
GROUP BY r.review_score, t.total_reviews
ORDER BY r.review_score;
```

17. Mostrar los clientes con pedidos entregados que nunca dejaron review.

Consulta SQL:

```
SELECT DISTINCT c.customer_unique_id
FROM olist_customers_dataset c
JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
WHERE o.order_status = 'delivered'
```

```
AND NOT EXISTS (
  SELECT 1 FROM olist_order_reviews_dataset r WHERE r.order_id = o.order_id
);
```

18. Mostrar el tiempo promedio entre aprobación del pago y entrega al transportista, por vendedor.

Consulta SQL:

```
SELECT oi.seller_id,
  ROUND(AVG(EXTRACT(EPOCH FROM (o.order_delivered_carrier_date - o.order_approved_at)) / 86400), 1) AS dias_preparacion
FROM olist_orders_dataset o
JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
WHERE o.order_delivered_carrier_date IS NOT NULL AND o.order_approved_at IS NOT NULL
GROUP BY oi.seller_id
ORDER BY dias_preparacion DESC;
```

19. Mostrar los pedidos con más de 3 productos distintos.

Consulta SQL:

```
SELECT oi.product_id, oi.price, t.product_category_name_english
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t ON p.product_category_name = t.product_category_name
WHERE oi.price > (
  SELECT AVG(oi2.price)
  FROM olist_order_items_dataset oi2
  JOIN olist_products_dataset p2 ON oi2.product_id = p2.product_id
  WHERE p2.product_category_name = p.product_category_name
)
ORDER BY oi.price DESC;
```

20. Pedidos con más de 3 productos distintos.

Consulta SQL:

```
SELECT order_id, COUNT(DISTINCT product_id) AS productos_distintos
FROM olist_order_items_dataset
GROUP BY order_id
HAVING COUNT(DISTINCT product_id) > 3
ORDER BY productos_distintos DESC;
```

21. Cantidad de pedidos por día de la semana

Consulta SQL:

```
SELECT TO_CHAR(order_purchase_timestamp, 'Day') AS dia_semana,
  EXTRACT(DOW FROM order_purchase_timestamp) AS dow,
  COUNT(*) AS cantidad_pedidos
FROM olist_orders_dataset
GROUP BY dia_semana, dow
ORDER BY dow;
```

22. Mostrar clientes cuyo primer pedido fue cancelado (subconsulta con MIN).

Consulta SQL:

```
WITH primer_pedido AS (
  SELECT c.customer_unique_id, o.order_id, o.order_status,
    ROW_NUMBER() OVER (PARTITION BY c.customer_unique_id ORDER BY
o.order_purchase_timestamp) AS rn
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
)
SELECT customer_unique_id, order_id
FROM primer_pedido
WHERE rn = 1 AND order_status = 'canceled';
```

23. Mostrar la categoría más vendida (por cantidad) de cada vendedor.

Consulta SQL:

```
WITH ventas_categoria AS (
  SELECT oi.seller_id, t.product_category_name_english AS categoria, COUNT(*) AS cantidad
  FROM olist_order_items_dataset oi
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
  GROUP BY oi.seller_id, t.product_category_name_english
)
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY seller_id ORDER BY cantidad DESC) AS rn
  FROM ventas_categoria
) x
WHERE rn = 1;
```

24. Mostrar el % de reviews con comentario vacío vs con comentario, por review_score.

Consulta SQL:

```
SELECT review_score,
  ROUND(100.0 * SUM(CASE WHEN review_comment_message IS NULL THEN 1 ELSE 0 END) /
COUNT(*), 2) AS pct_sin_comentario,
```

```
  ROUND(100.0 * SUM(CASE WHEN review_comment_message IS NOT NULL THEN 1 ELSE 0 END) /
COUNT(*), 2) AS pct_con_comentario
FROM olist_order_reviews_dataset
GROUP BY review_score
ORDER BY review_score;
```

25. Mostrar el top 5 productos con mayor diferencia entre precio mínimo y máximo vendido

Consulta SQL:

```
SELECT product_id, MIN(price) AS precio_min, MAX(price) AS precio_max,
  ROUND(MAX(price) - MIN(price), 2) AS diferencia
FROM olist_order_items_dataset
GROUP BY product_id
HAVING COUNT(*) > 1
ORDER BY diferencia DESC
LIMIT 5;
```

Nivel 4 - Experto

Consultas a practicar: Window functions avanzadas, NTILE, percentiles, RFM, cohortes, recursividad

1. ***Rankear a los vendedores dentro de cada estado según su facturación total, usando RANK().***

Consulta SQL:

```
SELECT s.seller_state, s.seller_id, ROUND(SUM(oi.price), 2) AS facturacion,
  RANK() OVER (PARTITION BY s.seller_state ORDER BY SUM(oi.price) DESC) AS ranking
FROM olist_order_items_dataset oi
JOIN olist_sellers_dataset s ON oi.seller_id = s.seller_id
GROUP BY s.seller_state, s.seller_id
ORDER BY s.seller_state, ranking;
```

2. ***Obtener el top 3 de productos más vendidos (por cantidad) dentro de cada categoría, usando ROW_NUMBER().***

Consulta SQL:

```
WITH ventas_producto AS (
  SELECT t.product_category_name_english AS categoria, oi.product_id,
       COUNT(*) AS cantidad_vendida
  FROM olist_order_items_dataset oi
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t
    ON p.product_category_name = t.product_category_name
  GROUP BY t.product_category_name_english, oi.product_id
)
SELECT categoria, product_id, cantidad_vendida, fila
FROM (
  SELECT categoria, product_id, cantidad_vendida,
    ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY cantidad_vendida DESC) AS fila
  FROM ventas_producto
) rankeado
WHERE fila <= 3
ORDER BY categoria, fila;
```

3. ***Hacer un análisis RFM simplificado: para cada cliente, calcular Recencia (días desde su última compra hasta la fecha máxima del dataset), Frecuencia (cantidad de pedidos) y Valor Monetario (total gastado).***

Consulta SQL:

```
WITH ultima_fecha AS (
  SELECT MAX(order_purchase_timestamp) AS fecha_max
  FROM olist_orders_dataset
),
pedidos_cliente AS (
```

```
  SELECT c.customer_unique_id,
        MAX(o.order_purchase_timestamp) AS ultima_compra,
        COUNT(DISTINCT o.order_id) AS frecuencia,
        SUM(pay.payment_value) AS valor_monetario
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY c.customer_unique_id
)
SELECT pc.customer_unique_id,
      EXTRACT(DAY FROM (uf.fecha_max - pc.ultima_compra)) AS recencia_dias,
      pc.frecuencia,
      ROUND(pc.valor_monetario, 2) AS valor_monetario
FROM pedidos_cliente pc, ultima_fecha uf
ORDER BY valor_monetario DESC;
```

Nota: *Este es el punto de partida típico para una segmentación RFM (Recencia, Frecuencia, Monto) en un análisis de clientes*

4. ***Armar un análisis de cohortes: agrupar a los clientes según el mes de su primera compra, y ver cuántos siguieron comprando en los meses siguientes (mes 0, mes 1, mes 2, etc.). (Cohortes)***

Consulta SQL:

```
WITH primera_compra AS (
  SELECT c.customer_unique_id,
        MIN(DATE_TRUNC('month', o.order_purchase_timestamp)) AS mes_cohorte
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  GROUP BY c.customer_unique_id
),
compras AS (
  SELECT c.customer_unique_id,
        DATE_TRUNC('month', o.order_purchase_timestamp) AS mes_compra
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
),
cohortes AS (
  SELECT pc.mes_cohorte, co.mes_compra, pc.customer_unique_id,
    (EXTRACT(YEAR FROM co.mes_compra) - EXTRACT(YEAR FROM pc.mes_cohorte)) * 12 +
    (EXTRACT(MONTH FROM co.mes_compra) - EXTRACT(MONTH FROM pc.mes_cohorte)) AS
mes_relativo
  FROM primera_compra pc
  JOIN compras co ON pc.customer_unique_id = co.customer_unique_id
)
```

```
SELECT mes_cohorte, mes_relativo, COUNT(DISTINCT customer_unique_id) AS clientes
FROM cohortes
GROUP BY mes_cohorte, mes_relativo
ORDER BY mes_cohorte, mes_relativo;
```

5. ***Usando LAG(), calcular la cantidad de días transcurridos entre pedidos consecutivos de un mismo cliente (para medir su recurrencia de compra).***

Consulta SQL:

```
WITH pedidos_ordenados AS (
  SELECT c.customer_unique_id, o.order_id, o.order_purchase_timestamp,
    LAG(o.order_purchase_timestamp) OVER (
      PARTITION BY c.customer_unique_id ORDER BY o.order_purchase_timestamp
    ) AS compra_anterior
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
)
SELECT customer_unique_id, order_id, order_purchase_timestamp,
  EXTRACT(DAY FROM (order_purchase_timestamp - compra_anterior)) AS
dias_desde_ultima_compra
FROM pedidos_ordenados
WHERE compra_anterior IS NOT NULL
ORDER BY customer_unique_id, order_purchase_timestamp;
```

6. ***Calcular qué porcentaje representa cada categoría de producto sobre la facturación total, usando una función de ventana (SUM() OVER()) en lugar de una subconsulta aparte.***

Consulta SQL:

```
SELECT t.product_category_name_english,
  ROUND(SUM(oi.price), 2) AS facturacion_categoria,
  ROUND(100.0 * SUM(oi.price) / SUM(SUM(oi.price)) OVER (), 2) AS porcentaje_del_total
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY facturacion_categoria DESC;
```

7. ***Construir una tabla RFM por cliente. Mostrar: customer_unique_id, Recency, Frequency, Monetary***

Consulta SQL:

```
SELECT t.product_category_name_english,
  ROUND(SUM(oi.price), 2) AS facturacion_categoria,
```

```
  ROUND(100.0 * SUM(oi.price) / SUM(SUM(oi.price)) OVER (), 2) AS porcentaje_del_total
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t
  ON p.product_category_name = t.product_category_name
GROUP BY t.product_category_name_english
ORDER BY facturacion_categoria DESC;
```

8. Percentil 90 del monto de pedidos, marcando outliers.

Consulta SQL:

```
WITH monto_por_pedido AS (
  SELECT order_id, SUM(payment_value) AS monto_total
  FROM olist_order_payments_dataset GROUP BY order_id
),
percentil AS (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY monto_total) AS p90
  FROM monto_por_pedido
)
SELECT mp.order_id, mp.monto_total, pe.p90,
  CASE WHEN mp.monto_total > pe.p90 THEN TRUE ELSE FALSE END AS es_outlier_alto
FROM monto_por_pedido mp, percentil pe
ORDER BY mp.monto_total DESC;
```

9. Media móvil de 3 meses de la facturación total (window frame).

Consulta SQL:

```
WITH ventas_mes AS (
  SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS mes, SUM(pay.payment_value) AS
ventas
  FROM olist_orders_dataset o
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY mes
)
SELECT mes, ROUND(ventas,2) AS ventas,
  ROUND(AVG(ventas) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)
AS media_movil_3m
FROM ventas_mes
ORDER BY mes;
```

10. Market basket: pares de categorías más compradas juntas (self-join)

Consulta SQL:

```
WITH categorias_por_pedido AS (
  SELECT DISTINCT oi.order_id, t.product_category_name_english AS categoria
  FROM olist_order_items_dataset oi
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
)
SELECT a.categoria AS categoria_1, b.categoria AS categoria_2, COUNT(*) AS veces_juntas
FROM categorias_por_pedido a
JOIN categorias_por_pedido b ON a.order_id = b.order_id AND a.categoria < b.categoria
GROUP BY a.categoria, b.categoria
ORDER BY veces_juntas DESC
LIMIT 15;
```

11. Primer vs. último pedido de cada cliente (FIRST_VALUE / LAST_VALUE).

Consulta SQL:

```
WITH pedidos_cliente AS (
  SELECT c.customer_unique_id, o.order_id, o.order_purchase_timestamp, SUM(pay.payment_value)
AS monto
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY c.customer_unique_id, o.order_id, o.order_purchase_timestamp
)
SELECT DISTINCT customer_unique_id,
  FIRST_VALUE(monto) OVER (PARTITION BY customer_unique_id ORDER BY
order_purchase_timestamp
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
monto_primer_pedido,
  LAST_VALUE(monto) OVER (PARTITION BY customer_unique_id ORDER BY
order_purchase_timestamp
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
monto_ultimo_pedido
FROM pedidos_cliente
ORDER BY customer_unique_id;
```

Ojo: sin el ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING, LAST_VALUE devuelve el valor de la fila actual, no el último real.

12. Tasa de retención mensual en % (cohortes llevadas a porcentaje)

Consulta SQL:

```
WITH primera_compra AS (
```

```
 SELECT c.customer_unique_id, MIN(DATE_TRUNC('month', o.order_purchase_timestamp)) AS
mes_cohorte
 FROM olist_customers_dataset c JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
 GROUP BY c.customer_unique_id
),
compras AS (
 SELECT c.customer_unique_id, DATE_TRUNC('month', o.order_purchase_timestamp) AS
mes_compra
 FROM olist_customers_dataset c JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
),
cohortes AS (
 SELECT pc.mes_cohorte, pc.customer_unique_id,
  (EXTRACT(YEAR FROM co.mes_compra) - EXTRACT(YEAR FROM pc.mes_cohorte)) * 12 +
  (EXTRACT(MONTH FROM co.mes_compra) - EXTRACT(MONTH FROM pc.mes_cohorte)) AS
mes_relativo
 FROM primera_compra pc JOIN compras co ON pc.customer_unique_id = co.customer_unique_id
),
tamano_cohorte AS (
 SELECT mes_cohorte, COUNT(DISTINCT customer_unique_id) AS clientes_iniciales
 FROM cohortes WHERE mes_relativo = 0 GROUP BY mes_cohorte
),
retencion AS (
 SELECT mes_cohorte, mes_relativo, COUNT(DISTINCT customer_unique_id) AS clientes
 FROM cohortes GROUP BY mes_cohorte, mes_relativo
)
SELECT r.mes_cohorte, r.mes_relativo, r.clientes, tc.clientes_iniciales,
 ROUND(100.0 * r.clientes / tc.clientes_iniciales, 1) AS retencion_pct
FROM retencion r JOIN tamano_cohorte tc ON r.mes_cohorte = tc.mes_cohorte
ORDER BY r.mes_cohorte, r.mes_relativo;
```

13. Serie completa de meses (incluso sin ventas) con generate_series.

Consulta SQL:

```
WITH meses AS (
 SELECT generate_series(
  (SELECT DATE_TRUNC('month', MIN(order_purchase_timestamp)) FROM olist_orders_dataset),
  (SELECT DATE_TRUNC('month', MAX(order_purchase_timestamp)) FROM olist_orders_dataset),
  '1 month'::interval
 ) AS mes
),
ventas_mes AS (
 SELECT DATE_TRUNC('month', order_purchase_timestamp) AS mes, COUNT(*) AS pedidos
 FROM olist_orders_dataset GROUP BY mes
```

```
)
SELECT m.mes, COALESCE(v.pedidos, 0) AS pedidos
FROM meses m LEFT JOIN ventas_mes v ON m.mes = v.mes
ORDER BY m.mes;
```

Comentario: generate_series es específico de PostgreSQL. Evita que un mes sin ventas directamente desaparezca del reporte.

14. Detección de "rachas" de compra (pedidos con menos de 30 días entre sí).

Consulta SQL:

```
WITH pedidos_ordenados AS (
  SELECT c.customer_unique_id, o.order_id, o.order_purchase_timestamp
  FROM olist_customers_dataset c JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
),
con_diferencia AS (
  SELECT po.*,
    EXTRACT(DAY FROM (po.order_purchase_timestamp -
      LAG(po.order_purchase_timestamp) OVER (PARTITION BY po.customer_unique_id ORDER BY
po.order_purchase_timestamp)
    )) AS dias_desde_anterior
  FROM pedidos_ordenados po
)
SELECT customer_unique_id, order_id, order_purchase_timestamp, dias_desde_anterior,
  CASE WHEN dias_desde_anterior <= 30 THEN 'racha activa' ELSE 'nueva racha' END AS estado
FROM con_diferencia
ORDER BY customer_unique_id, order_purchase_timestamp;
```

15. CTE recursiva: generar una serie de fechas "a mano" (sin generate_series) para practicar WITH RECURSIVE.

Consulta SQL:

```
WITH RECURSIVE serie_fechas AS (
  SELECT DATE_TRUNC('month', MIN(order_purchase_timestamp))::date AS fecha
  FROM olist_orders_dataset
  UNION ALL
  SELECT (fecha + INTERVAL '1 month')::date
  FROM serie_fechas
  WHERE fecha < (SELECT DATE_TRUNC('month', MAX(order_purchase_timestamp))::date FROM
olist_orders_dataset)
)
SELECT fecha FROM serie_fechas ORDER BY fecha;
```

Comentario: Esta es la forma "clásica" de recursividad en SQL: la CTE se referencia a sí misma, sumando un mes en cada vuelta, hasta cumplir la condición de corte en el WHERE. generate_series hace lo mismo pero ya resuelto por PostgreSQL.

16. Segmentar productos en deciles de precio (NTILE 10).

Consulta SQL:

```
SELECT product_id, price,
  NTILE(10) OVER (ORDER BY price) AS decil_precio
FROM olist_order_items_dataset
ORDER BY price;
```

17. Análisis ABC/Pareto: % acumulado de facturación para identificar qué vendedores generan el 80% de las ventas

Consulta SQL:

```
WITH ventas_seller AS (
  SELECT seller_id, SUM(price) AS ventas
  FROM olist_order_items_dataset
  GROUP BY seller_id
),
acumulado AS (
  SELECT seller_id, ventas,
    SUM(ventas) OVER (ORDER BY ventas DESC) AS acumulado,
    SUM(ventas) OVER () AS total_general
  FROM ventas_seller
)
SELECT seller_id, ventas,
  ROUND(100.0 * acumulado / total_general, 2) AS pct_acumulado
FROM acumulado
ORDER BY ventas DESC;
```

18. LEAD: detectar si el próximo pedido de un cliente cambia de categoría respecto al actual.

Consulta SQL:

```
WITH compras_categoria AS (
  SELECT c.customer_unique_id, o.order_purchase_timestamp,
    t.product_category_name_english AS categoria
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
```

```
)
SELECT customer_unique_id, order_purchase_timestamp, categoria,
  LEAD(categoria) OVER (PARTITION BY customer_unique_id ORDER BY order_purchase_timestamp)
AS categoria_siguiente,
  CASE WHEN LEAD(categoria) OVER (PARTITION BY customer_unique_id ORDER BY
order_purchase_timestamp) <> categoria
      THEN 'cambió de categoría' ELSE 'misma categoría' END AS comportamiento
FROM compras_categoria
ORDER BY customer_unique_id, order_purchase_timestamp;
```

19. Vendedores que compiten en la(s) misma(s) categoría(s) que el vendedor top 1 en facturación (self-join analítico).

Consulta SQL:

```
WITH ventas_seller AS (
  SELECT seller_id, SUM(price) AS ventas
  FROM olist_order_items_dataset GROUP BY seller_id
),
top_seller AS (
  SELECT seller_id FROM ventas_seller ORDER BY ventas DESC LIMIT 1
),
categorias_top AS (
  SELECT DISTINCT t.product_category_name_english AS categoria
  FROM olist_order_items_dataset oi
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
  WHERE oi.seller_id = (SELECT seller_id FROM top_seller)
)
SELECT DISTINCT oi.seller_id
FROM olist_order_items_dataset oi
JOIN olist_products_dataset p ON oi.product_id = p.product_id
JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
WHERE t.product_category_name_english IN (SELECT categoria FROM categorias_top)
AND oi.seller_id <> (SELECT seller_id FROM top_seller);
```

20. Evolución del gasto acumulado (lifetime value) de cada cliente, pedido a pedido.

Consulta SQL:

```
WITH pedidos_cliente AS (
  SELECT c.customer_unique_id, o.order_id, o.order_purchase_timestamp, SUM(pay.payment_value)
AS monto
```

```
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY c.customer_unique_id, o.order_id, o.order_purchase_timestamp
)
SELECT customer_unique_id, order_id, order_purchase_timestamp, monto,
  SUM(monto) OVER (
    PARTITION BY customer_unique_id ORDER BY order_purchase_timestamp
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS gasto_acumulado
FROM pedidos_cliente
ORDER BY customer_unique_id, order_purchase_timestamp;
```

21. Facturación mensual de cada vendedor comparada contra su propio promedio histórico.

Consulta SQL:

```
WITH ventas_mes_seller AS (
  SELECT oi.seller_id, DATE_TRUNC('month', o.order_purchase_timestamp) AS mes, SUM(oi.price) AS
ventas
  FROM olist_order_items_dataset oi
  JOIN olist_orders_dataset o ON oi.order_id = o.order_id
  GROUP BY oi.seller_id, mes
)
SELECT seller_id, mes, ventas,
  ROUND(AVG(ventas) OVER (PARTITION BY seller_id), 2) AS promedio_historico,
  ROUND(ventas - AVG(ventas) OVER (PARTITION BY seller_id), 2) AS diferencia_vs_promedio
FROM ventas_mes_seller
ORDER BY seller_id, mes;
```

22. Meses donde la facturación total cayó más de 20% respecto al mes anterior.

Consulta SQL:

```
WITH ventas_mes AS (
  SELECT DATE_TRUNC('month', o.order_purchase_timestamp) AS mes, SUM(pay.payment_value) AS
ventas
  FROM olist_orders_dataset o
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY mes
)
SELECT mes, ventas,
  LAG(ventas) OVER (ORDER BY mes) AS ventas_mes_anterior,
  ROUND(100.0 * (ventas - LAG(ventas) OVER (ORDER BY mes)) / LAG(ventas) OVER (ORDER BY mes),
2) AS variacion_pct
```

```
FROM ventas_mes
WHERE (ventas - LAG(ventas) OVER (ORDER BY mes)) / NULLIF(LAG(ventas) OVER (ORDER BY mes),0)
< -0.20;
```

Comentario: Nota el uso de NULLIF para evitar división por cero si algún mes anterior tuviera 0 en ventas.

23. Categoría con mayor facturación por cada estado del cliente (una por estado).

Consulta SQL:

```
WITH ventas_categoria_estado AS (
  SELECT c.customer_state, t.product_category_name_english AS categoria, SUM(oi.price) AS ventas
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_items_dataset oi ON o.order_id = oi.order_id
  JOIN olist_products_dataset p ON oi.product_id = p.product_id
  JOIN product_category_name_translation t ON p.product_category_name =
t.product_category_name
  GROUP BY c.customer_state, t.product_category_name_english
)
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_state ORDER BY ventas DESC) AS rn
  FROM ventas_categoria_estado
) x
WHERE rn = 1;
```

24. Vendedores "de una sola vez" (solo vendieron en un mes) vs. recurrentes

Consulta SQL:

```
WITH meses_activos AS (
  SELECT oi.seller_id, COUNT(DISTINCT DATE_TRUNC('month', o.order_purchase_timestamp)) AS
meses_con_ventas
  FROM olist_order_items_dataset oi
  JOIN olist_orders_dataset o ON oi.order_id = o.order_id
  GROUP BY oi.seller_id
)
SELECT seller_id, meses_con_ventas,
  CASE WHEN meses_con_ventas = 1 THEN 'one-hit wonder' ELSE 'recurrente' END AS tipo_vendedor
FROM meses_activos
ORDER BY meses_con_ventas DESC;
```

25. Segmentación RFM completa con etiquetas de negocio (Champions, Leales, En riesgo, Perdidos).

Consulta SQL:

```
WITH rfm_base AS (
```

```
  SELECT c.customer_unique_id,
    EXTRACT(DAY FROM ((SELECT MAX(order_purchase_timestamp) FROM olist_orders_dataset) -
MAX(o.order_purchase_timestamp))) AS recencia,
    COUNT(DISTINCT o.order_id) AS frecuencia,
    SUM(pay.payment_value) AS monetario
  FROM olist_customers_dataset c
  JOIN olist_orders_dataset o ON c.customer_id = o.customer_id
  JOIN olist_order_payments_dataset pay ON o.order_id = pay.order_id
  GROUP BY c.customer_unique_id
),
rfm_score AS (
  SELECT *,
    NTILE(5) OVER (ORDER BY recencia ASC) AS r_score,
    NTILE(5) OVER (ORDER BY frecuencia DESC) AS f_score,
    NTILE(5) OVER (ORDER BY monetario DESC) AS m_score
  FROM rfm_base
)
SELECT customer_unique_id, recencia, frecuencia, ROUND(monetario,2) AS monetario,
  r_score, f_score, m_score,
  CASE
    WHEN r_score >= 4 AND f_score >= 4 AND m_score >= 4 THEN 'Champion'
    WHEN r_score >= 3 AND f_score >= 3 THEN 'Cliente leal'
    WHEN r_score <= 2 AND f_score >= 3 THEN 'En riesgo'
    WHEN r_score <= 2 AND f_score <= 2 THEN 'Perdido'
    ELSE 'Regular'
  END AS segmento
FROM rfm_score
ORDER BY monetario DESC;
```